

Análise Assintótica de Algoritmos Recursivos

Algoritmos de divisão e conquista

Algoritmos recursivos - Ex.: Merge Sort

```
1 def merge_sort(lista):  
2     if len(lista) <= 1:  
3         return lista  
4  
5     meio = len(lista) // 2  
6  
7     esquerda = merge_sort(lista[:meio])  
8     direita = merge_sort(lista[meio:])  
9  
10    return merge(esquerda, direita)
```

As instruções que não são chamadas de funções são executadas uma vez.

O total de instruções da chamada à função merge depende da complexidade desta função.

Algoritmos recursivos - Ex.: Merge Sort

```
1 def merge(esquerda, direita):  
2     resultado = []  
3     i = j = 0  
4  
5     while i < len(esquerda) and j < len(direita):  
6         if esquerda[i] <= direita[j]:  
7             resultado.append(esquerda[i])  
8             i += 1  
9         else:  
10            resultado.append(direita[j])  
11            j += 1  
12  
13    resultado.extend(esquerda[i:])  
14    resultado.extend(direita[j:])  
15  
16    return resultado
```

No pior caso, a repetição faz i e j percorrem todos os elementos de esquerda e direita. Como cada lista tem tamanho $n/2$, o total de instruções executadas por merge é $\Theta(n)$.

Algoritmos recursivos - Ex.: Merge Sort

```
1 def merge_sort(lista):  
2     if len(lista) <= 1:  
3         return lista  
4  
5     meio = len(lista) // 2  
6  
7     esquerda = merge_sort(lista[:meio])  
8     direita = merge_sort(lista[meio:])  
9  
10    return merge(esquerda, direita)
```

Obs.: Fatiamento em Python é $O(n)$, mas não altera a complexidade pois a chamada merge também é $O(n)$. É possível fazer em tempo $O(1)$ passando posição inicial e final.

Como contar o número de execuções nas instruções onde há chamadas recursivas?

Algoritmos recursivos de divisão e conquista

Divisão e conquista

- 1 Divide a entrada (tamanho n) em b partes c/ aproximadamente o mesmo tamanho (n/b).
- 2 Resolve recursivamente uma quantidade “ a ” destas partes.
- 3 Combina as soluções das partes para formar a solução do problema original.

Assim, o total $T(n)$ de instruções executadas é descrito pela recorrência:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

onde:

- a é o número de partes,
- n/b é o tamanho de cada parte,
- $f(n)$ é o tempo gasto na divisão do problema e na combinação das soluções.

Algoritmos recursivos de divisão e conquista - Ex.: Merge Sort

O algoritmo de Merge Sort divide a entrada em 2 partes ($b = 2$), resolve recursivamente as duas partes ($a = 2$) e gasta tempo $\Theta(n)$ para combinar as soluções das partes (função merge).

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \Theta(n)$$

$$T(n) \in \Theta(?)$$

Simplificação Θ de recorrências de divisão e conquista

Se $T(n) = a \cdot T(n/b) + f(n)$, com $f(n) = \Theta(n^d \log^e n)$,

$\log_b a$ versus d	e	Solução
$<$	qualquer	$\Theta(f(n))$
$=$	> -1	$\Theta(f(n) \cdot \log n)$
$=$	$= -1$	$\Theta(n^d \cdot \log \log n)$
$=$	< -1	$\Theta(n^{\log_b a})$
$>$	qualquer	$\Theta(n^{\log_b a})$

Exemplos:

- 1 $T(n) = 9T(n/3) + n^3.$
- 2 $T(n) = 9T(n/3) + n^2.$
- 3 $T(n) = 9T(n/3) + n^2 / \log n.$
- 4 $T(n) = 9T(n/3) + n^2 / \log^2 n.$
- 5 $T(n) = 9T(n/3) + n.$
- 6 Mergesort: $T(n) = 2T(n/2) + \Theta(n).$

Simplificação Θ de recorrências de divisão e conquista

Se $T(n) = a \cdot T(n/b) + f(n)$, com $f(n) = \Theta(n^d \log^e n)$,

$\log_b a$ versus d	e	Solução
$<$	qualquer	$\Theta(f(n))$
$=$	> -1	$\Theta(f(n) \cdot \log n)$
$=$	$= -1$	$\Theta(n^d \cdot \log \log n)$
$=$	< -1	$\Theta(n^{\log_b a})$
$>$	qualquer	$\Theta(n^{\log_b a})$

Exemplos:

- 1 $T(n) = 9T(n/3) + n^3 \in \Theta(n^3)$.
- 2 $T(n) = 9T(n/3) + n^2 \in \Theta(n^2 \log n)$.
- 3 $T(n) = 9T(n/3) + n^2 / \log n \in \Theta(n^2 \log \log n)$.
- 4 $T(n) = 9T(n/3) + n^2 / \log^2 n \in \Theta(n^2)$.
- 5 $T(n) = 9T(n/3) + n \in \Theta(n^2)$.
- 6 Mergesort: $T(n) = 2T(n/2) + \Theta(n) \in \Theta(n \log n)$.

Exercícios

Exercício: Solução de recorrências em notação Θ

Para cada recorrência de divisão e conquista da forma $T(n) = a \cdot T(n/b) + f(n)$, com $f(n) = \Theta(n^d \log^e n)$, determine a sua solução em notação Θ :

❶ $T(n) = 2T(n/2) + n^2$

❷ $T(n) = 8T(n/2) + n^3$

❸ $T(n) = 4T(n/2) + \frac{n^2}{\log n}$

❹ $T(n) = 4T(n/2) + \frac{n^2}{\log^2 n}$

❺ $T(n) = 16T(n/2) + n^2$

❻ $T(n) = 4T(n/2) + n^3 \log n$

❼ $T(n) = 2T(n/2) + n \log^2 n$

❽ $T(n) = T(n/2) + 1$ (busca binária)

Gabarito: Solução de recorrências em notação Θ

- ① $T(n) \in \Theta(n^2)$
 $\log_2 2 = 1 < d = 2 \implies \Theta(f(n)).$
- ② $T(n) \in \Theta(n^3 \log n)$
 $\log_2 8 = 3 = d, e = 0 > -1 \implies \Theta(f(n) \cdot \log n).$
- ③ $T(n) \in \Theta(n^2 \log \log n)$
 $\log_2 4 = 2 = d, e = -1 \implies \Theta(n^d \cdot \log \log n).$
- ④ $T(n) \in \Theta(n^2)$
 $\log_2 4 = 2 = d, e = -2 < -1 \implies \Theta(n^{\log_b a}).$
- ⑤ $T(n) \in \Theta(n^4)$
 $\log_2 16 = 4 > d = 2 \implies \Theta(n^{\log_b a}).$
- ⑥ $T(n) \in \Theta(n^3 \log n)$
 $\log_2 4 = 2 < d = 3 \implies \Theta(f(n)).$
- ⑦ $T(n) \in \Theta(n \log^3 n)$
 $\log_2 2 = 1 = d, e = 2 > -1 \implies \Theta(f(n) \cdot \log n).$
- ⑧ $T(n) \in \Theta(\log n)$
 $\log_2 1 = 0 = d, e = 0 \implies \Theta(f(n) \cdot \log n) = \Theta(\log n).$

Exercício: Recorrência de Divisão e Conquista

Considere o algoritmo abaixo, que encontra a soma do subvetor contíguo de valor máximo em um vetor $A[esq..dir]$ com $n = dir - esq + 1$ elementos (assuma que a função `max_cruzado` executa em tempo $\Theta(n \log n)$, embora seja possível em $O(n)$):

```
1 def max_subvetor(A, esq, dir):  
2     if esq == dir:  
3         return A[esq]  
4     meio = (esq + dir) // 2  
5     max_esq = max_subvetor(A, esq, meio)  
6     max_dir = max_subvetor(A, meio + 1, dir)  
7     max_cruz = max_cruzado(A, esq, meio, dir)  
8     return max(max_esq, max_dir, max_cruz)
```

Questão:

- 1 Monte a equação de recorrência $T(n)$ para o tempo de execução do algoritmo.
- 2 Determine a solução assintótica da recorrência em notação Θ .

Gabarito: Recorrência de Divisão e Conquista

1. Equação de Recorrência

Para um vetor de tamanho $n = \text{dir} - \text{esq} + 1$:

- Caso base ($n = 1$): $T(1) = \Theta(1)$
- Passo recursivo ($n > 1$): O algoritmo realiza 2 chamadas recursivas sobre metades do vetor ($\approx n/2$) e executa a função `max_cruzado` em tempo $\Theta(n \log n)$.

$$T(n) = 2 \cdot T(n/2) + \Theta(n \log n)$$

2. Solução em Notação Θ

Analisando a forma $T(n) = a \cdot T(n/b) + f(n)$, onde $f(n) = \Theta(n^d \log^e n)$:

- Parâmetros: $a = 2$, $b = 2$, e $f(n) = \Theta(n \log n) \implies d = 1, e = 1$.
- Comparação: $\log_b a = \log_2 2 = 1 = d$.
- Como $\log_b a = d$ e $e = 1 > -1$, aplica-se a regra:

$$T(n) \in \Theta(f(n) \cdot \log n) = \Theta(n \cdot \log^2 n)$$